GRUPO 1 · REQUERIMIENTOS FUNCIONALES

Autenticación y Acceso

Módulo: Login / Logout / Recuperación de contraseña

Curso de Automatización de Pruebas de Software

Universidad Nacional de Asunción — CIT

Profesor: Steven Gracia

Campo	Valor
Documento	Requerimientos funcionales — Grupo 1
Versión	1.0
Fecha	22 de agosto de 2026
Sistemas alcanzados	aiquaa-sandbox-api (REST) y aiquaa-sandbox-web (front)
Fuente de la lógica	Código fuente de ambos repositorios y scripts SQL del schema qa_training
Uso previsto	Base para escribir escenarios BDD/Gherkin y casos de prueba automatizados

> Documento derivado del código fuente de aiquaa-sandbox-api y aiquaa-sandbox-web. Describe el comportamiento realmente implementado en el sandbox de práctica, no un sistema bancario productivo. El anexo normativo es material de referencia orientativo.

Contenido

1. Alcance y actores
2. Precondiciones y acceso
3. Modelo de datos del módulo
4. Requerimientos funcionales
5. Reglas transversales de la API
6. Flujo en la aplicación web
7. Criterios de aceptación
8. Anexo normativo (BCP / SEDECO) y brechas

1. Alcance y actores

Cubre el ingreso y la salida de un usuario de negocio del sandbox y el circuito de recuperación de acceso. El sandbox no almacena contraseñas: la identidad se prueba con el email de un usuario existente y activo, y cada evento del circuito queda registrado como una fila de bitácora en la tabla `sesiones`. El módulo agrega además un recurso CRUD completo (`GET` / `POST` / `PUT` / `DELETE`) sobre esa misma tabla, pensado como ejemplo didáctico de las cuatro operaciones, separado del flujo realista de `/auth/*`.

Fuera de alcance: Verificación de contraseña, tokens de sesión, expiración, segundo factor y bloqueo por intentos fallidos: ninguno está implementado. La autenticación del canal es la API key, no el login de usuario.

Actores

Actor	Descripción
Alumno / QA automatizador	Consume la API con su propia API key y automatiza escenarios BDD sobre el módulo de su grupo.
Usuario de negocio (fila de `usuarios`)	Identidad de dominio sobre la que operan los endpoints: es dueño de cuentas, facturas, tarjetas, órdenes, reservas, notificaciones y roles.
API REST (aiquaa-sandbox-api)	Ejecuta la lógica de negocio con SQL fijo bajo el rol de base de datos `qa_api` (SELECT + INSERT + UPDATE, sin DELETE).
Aplicación web (aiquaa-sandbox-web)	Interfaz que consume la API a través del proxy `/api/proxy/{path}`; nunca llama al backend directamente desde el navegador.

Endpoints del módulo

Método y ruta	Requerimiento	Descripción
POST /api/v1/auth/login	RF-G1-01	Valida el email de un usuario activo y registra el evento de login.
POST /api/v1/auth/logout	RF-G1-02	Registra el evento de cierre de sesión de un usuario.
POST /api/v1/auth/forgot-password	RF-G1-03	Registra la solicitud de recuperación de contraseña.
POST /api/v1/auth/reset-password	RF-G1-04	Registra el reseteo de contraseña como completado.
GET /api/v1/sesiones	RF-G1-05	Lista eventos de sesión vigentes, opcionalmente por titular.
POST /api/v1/sesiones	RF-G1-06	Crea un evento de sesión genérico (cualquier tipo/resultado).
GET /api/v1/sesiones/{id}	RF-G1-07	Devuelve un evento de sesión puntual.
PUT /api/v1/sesiones/{id}	RF-G1-08	Reemplaza los metadatos de contexto (`ip`, `userAgent`) de un evento.
DELETE /api/v1/sesiones/{id}	RF-G1-09	Da de baja (soft-delete) un evento de sesión.

2. Precondiciones y acceso

- Toda request a la API debe enviar el header `x-api-key` con una clave existente y con `active = true` en la tabla `public.api_keys`; sin header o con clave inválida/inactiva la respuesta es `401 UNAUTHORIZED` con el mensaje genérico "Invalid or inactive API key.".
- Cada API key admite un máximo de **200 requests por minuto** (ventana deslizante). Superado el límite la API responde `429 RATE_LIMITED` con los headers `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset`.
- Los datos de práctica viven en el schema aislado `qa_training` y se cargan con `scripts/seed-data.sql`; la data es determinística, por lo que un escenario puede apoyarse en los registros sembrados.
- La API no expone borrado físico: el rol `qa_api` no tiene permiso `DELETE` en Postgres, y todo endpoint `DELETE` de la API hace baja lógica (`activo = false`), devolviendo `204 No Content` sin cuerpo.
- Casi todas las tablas de negocio (`sesiones`, `transferencias`, `facturas`, `tarjetas`, `notificaciones`, `ordenes`, `reservas`, `movimientos`, `roles`, además de `usuarios` y `cuentas` que ya la tenían) tienen columna `activo` / `activa`; todo `GET`, `PUT` y `DELETE` de esa tabla filtra u opera solo sobre filas con `activo = true` — una fila dada de baja deja de ser visible y de poder reemplazarse, pero no se borra.
- En la aplicación web el acceso tiene dos capas: capa 1 la API key (pantalla `/login`, se valida contra `GET /api/v1/roles` antes de guardarse) y capa 2 el usuario de negocio (pantalla `/auth/login`). Con `NEXT_PUBLIC_DEMO_MODE=true` la capa 1 se omite y el proxy inyecta una key demo del servidor.
- Las rutas `/usuarios/*` de la web son la excepción al guard de capa 2: se pueden usar solo con la API key, porque son el punto de partida para obtener un `usuarioId`.
- Los escenarios de login necesitan al menos un usuario con `activo = true` y otro con `activo = false` para cubrir el camino negativo; ambos existen en los datos sembrados.
- Los endpoints de logout y de reset completado identifican al usuario por `usuarioId` numérico, no por email.
- El CRUD de `sesiones` (RF-G1-05 a RF-G1-09) es un recurso aparte del flujo `/auth/*`: crear o editar un evento con `POST` / `PUT` no inicia ni cierra ninguna sesión real, solo escribe/edita la fila de bitácora directamente.

3. Modelo de datos del módulo

usuarios

Identidad de negocio. Este módulo solo lee de esta tabla; nunca la modifica.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`nombre`	`text`, obligatorio
`email`	`text`, obligatorio y único
`activo`	`boolean`, obligatorio, por defecto `true`
`kyc_estado`	`text`, uno de `pendiente` / `verificado` / `rechazado`
`created_at`	`timestamptz`, por defecto `now()`

sesiones

Bitácora de eventos de acceso. Es el resultado observable de los cuatro requerimientos del módulo.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`usuario_id`	`bigint`, obligatorio, referencia a `usuarios(id)`
`tipo_evento`	`text`, obligatorio, uno de `login` / `logout` / `password_reset_solicitado` / `password_reset_completado`
`exitoso`	`boolean`, obligatorio, por defecto `true`
`ip`	`text`, opcional; en `/auth/*` la API la toma de los headers de la request
`user_agent`	`text`, opcional; `/auth/*` no lo completa, pero `POST/PUT /sesiones` sí lo aceptan
`created_at`	`timestamptz`, por defecto `now()`
`activo`	`boolean`, obligatorio, por defecto `true`; lo usa el `DELETE` del CRUD (RF-G1-09)

> Los cuatro endpoints de `/auth/*` solo escriben filas con `exitoso = true`: un intento de login rechazado devuelve error pero **no** deja registro en `sesiones`. El CRUD genérico (`POST /sesiones`, RF-G1-06) no tiene esa restricción: acepta `exitoso=false` y cualquier `tipoEvento` de la lista, sin haber ocurrido el evento real — útil para sembrar datos de prueba, pero no confundirlo con una bitácora de auditoría fiable.

4. Requerimientos funcionales

RF-G1-01 — Iniciar sesión

`POST /api/v1/auth/login`

Permite iniciar sesión en el sandbox indicando el email de un usuario. Al ser aceptado, el sistema devuelve la identidad del usuario y registra el ingreso en la bitácora de sesiones.

Entradas y validaciones

- `email` (obligatorio): texto con formato de correo electrónico válido. Un valor sin formato de email es rechazado antes de consultar la base de datos.

Reglas de negocio

- El sistema busca un único usuario cuyo `email` coincida exactamente con el enviado (la comparación distingue mayúsculas y minúsculas y no recorta espacios).
- El ingreso solo se acepta si el usuario existe **y** tiene `activo = true`.
- Un usuario inexistente y un usuario inactivo producen exactamente el mismo error, con el mismo mensaje: el sistema no revela cuál de las dos situaciones ocurrió.
- El rechazo se comunica como error de validación (HTTP 400), no como error de autenticación (HTTP 401), para no confundirlo con un problema de la API key.
- Aceptado el ingreso, se inserta una fila en `sesiones` con `tipo_evento = 'login'`, `exitoso = true` y la IP de origen de la request.

Respuesta esperada

- `200 OK` con `data` conteniendo `id`, `nombre`, `email` y `activo` del usuario. No se devuelve token alguno.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	El email no existe o el usuario está inactivo (mensaje: "Usuario no encontrado o inactivo.").
`VALIDATION_ERROR`	400	Falta el campo `email` o no tiene formato de correo válido.
`UNAUTHORIZED`	401	Falta la API key o es inválida.

Fuente: `app/api/v1/auth/login/route.ts`

RF-G1-02 — Cerrar sesión

`POST /api/v1/auth/logout`

Registra el cierre de sesión de un usuario identificado por su número interno. Es la contraparte del login en la bitácora.

Entradas y validaciones

- `usuarioId` (obligatorio): número entero positivo. Se acepta también como texto numérico, que el sistema convierte.

Reglas de negocio

- El sistema verifica que exista un usuario con ese identificador; a diferencia del login, **no** exige que esté activo.
- Si el usuario existe, inserta una fila en `sesiones` con `tipo_evento = 'logout'`, `exitoso = true` y la IP de origen.
- La operación puede repetirse: cada llamada agrega una fila nueva a la bitácora, sin estado que impida el segundo cierre.

Respuesta esperada

- `200 OK` con `data` conteniendo la fila de `sesiones` recién creada (incluye `id`, `usuario_id`, `tipo_evento`, `exitoso`, `ip` y `created_at`).

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un usuario con ese `usuarioId` (mensaje: "Usuario no encontrado.").
`VALIDATION_ERROR`	400	`usuarioId` ausente, cero, negativo o no numérico.

Fuente: `app/api/v1/auth/logout/route.ts`

RF-G1-03 — Solicitar recuperación de contraseña

`POST /api/v1/auth/forgot-password`

Registra que un usuario solicitó recuperar su acceso. El sandbox no genera ni envía ningún token: la solicitud queda asentada en la bitácora y habilita el paso de confirmación.

Entradas y validaciones

- `email` (obligatorio): texto con formato de correo electrónico válido.

Reglas de negocio

- El sistema busca al usuario por email exacto.
- A diferencia del login, este endpoint **no** verifica que el usuario esté activo: un usuario inactivo puede solicitar la recuperación.
- Si el usuario no existe, la respuesta es un 404 explícito — es decir, el endpoint sí revela si un email está registrado.
- Si existe, se inserta una fila en `sesiones` con `tipo_evento = 'password_reset_solicitado'`, `exitoso = true` y la IP de origen.
- La fila devuelta contiene el `usuario_id`, que es el dato con el que después se confirma el reseteo (RF-G1-04).

Respuesta esperada

- `200 OK` con `data` conteniendo la fila de `sesiones` creada.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	El email no corresponde a ningún usuario (mensaje: "Usuario no encontrado.").
`VALIDATION_ERROR`	400	Falta el campo `email` o no tiene formato de correo válido.

Fuente: `app/api/v1/auth/forgot-password/route.ts`

RF-G1-04 — Completar el reseteo de contraseña

`POST /api/v1/auth/reset-password`

Cierra el circuito de recuperación registrando que el reseteo se completó. No cambia ninguna credencial, porque el sandbox no las almacena.

Entradas y validaciones

- `usuarioId` (obligatorio): número entero positivo.

Reglas de negocio

- El sistema verifica que el usuario exista; no exige que esté activo.
- No verifica que exista una solicitud previa (`password_reset_solicitado`): el reseteo puede completarse sin haberlo pedido.
- No hay token, ni verificación de vigencia, ni un solo uso: la operación puede repetirse tantas veces como se llame, agregando una fila por vez.
- Se inserta una fila en `sesiones` con `tipo_evento = 'password_reset_completado'`, `exitoso = true` y la IP de origen.

Respuesta esperada

- `200 OK` con `data` conteniendo la fila de `sesiones` creada.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un usuario con ese `usuarioId` (mensaje: "Usuario no encontrado.").
`VALIDATION_ERROR`	400	`usuarioId` ausente, cero, negativo o no numérico.

Fuente: `app/api/v1/auth/reset-password/route.ts`

RF-G1-05 — Listar eventos de sesión

`GET /api/v1/sesiones`

Devuelve los eventos de sesión vigentes del sandbox, con la posibilidad de acotar el resultado a un titular determinado. Es el recurso CRUD genérico que expone la misma tabla que alimentan los cuatro endpoints de `/auth/*`.

Entradas y validaciones

- `usuarioId` (opcional, por query): número entero positivo.

Reglas de negocio

- Devuelve únicamente eventos con `activo = true`: uno dado de baja con RF-G1-09 deja de aparecer.
- Con `usuarioId`, devuelve todos los eventos de ese titular ordenados por identificador ascendente; sin filtro, los primeros 100 eventos del sandbox.
- Incluye tanto los eventos creados por `/auth/*` como los creados directamente con RF-G1-06.

Respuesta esperada

- `200 OK` con `data` como arreglo de eventos, cada uno con `id`, `usuario_id`, `tipo_evento`, `exitoso`, `ip`, `user_agent`, `created_at` y `activo`.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	`usuarioId` presente pero no numérico, cero o negativo.
`UNAUTHORIZED`	401	Falta la API key o es inválida.

Fuente: `app/api/v1/sesiones/route.ts`

RF-G1-06 — Crear un evento de sesión genérico

`POST /api/v1/sesiones`

Inserta directamente una fila en la bitácora de sesiones, sin pasar por el flujo de `/auth/*`. Pensado como ejemplo didáctico de creación vía CRUD, no como parte del circuito de autenticación real.

Entradas y validaciones

- `usuarioId` (obligatorio): número entero positivo.
- `tipoEvento` (obligatorio): uno de `login`, `logout`, `password_reset_solicitado`, `password_reset_completado`.
- `exitoso` (opcional): exactamente el texto `true` o `false`; si se omite, la fila se crea con `true`.
- `ip` (opcional): texto libre.
- `userAgent` (opcional): texto libre.

Reglas de negocio

- A diferencia de `/auth/login` y compañía, este endpoint **no valida que el evento haya ocurrido de verdad**: acepta `exitoso=false` o cualquier combinación de campos sin que exista un login/logout real detrás.
- El titular debe existir: un `usuarioId` inexistente falla como error de ejecución por violación de clave foránea.
- La fila se crea siempre con `activo = true`.

Respuesta esperada

- `201 Created` con `data` conteniendo el evento creado.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta un campo obligatorio, `tipoEvento` tiene un valor fuera de la lista permitida, o el `usuarioId` indicado no existe.

Fuente: `app/api/v1/sesiones/route.ts`

RF-G1-07 — Consultar un evento de sesión

`GET /api/v1/sesiones/{id}`

Devuelve el detalle de un evento de sesión puntual.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Devuelve el evento cuyo identificador coincide exactamente, solo si `activo = true`.
- Un evento dado de baja responde igual que uno inexistente: 404.

Respuesta esperada

- `200 OK` con `data` conteniendo el evento completo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un evento vigente con ese identificador (mensaje: "Sesión no encontrada.").
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/sesiones/[id]/route.ts`

RF-G1-08 — Reemplazar los metadatos de un evento de sesión

`PUT /api/v1/sesiones/{id}`

Actualiza únicamente los metadatos de contexto (`ip`, `userAgent`) de un evento existente. El contenido del evento en sí (`usuarioId`, `tipoEvento`, `exitoso`) queda fijo: un evento es un registro de lo que pasó, no algo que deba poder cambiarse retroactivamente.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `ip` (opcional): texto libre; si se omite se guarda vacío.
- `userAgent` (opcional): texto libre; si se omite se guarda vacío.

Reglas de negocio

- Solo puede reemplazar un evento con `activo = true`; uno dado de baja responde 404.
- Omitir `ip` o `userAgent` en el cuerpo los deja vacíos (`null`), no los conserva: es un reemplazo completo, no un parche.

Respuesta esperada

- `200 OK` con `data` conteniendo el evento ya actualizado.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un evento vigente con ese identificador (mensaje: "Sesión no encontrada.").
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/sesiones/[id]/route.ts`

RF-G1-09 — Dar de baja un evento de sesión

`DELETE /api/v1/sesiones/{id}`

Da de baja lógica un evento de sesión, quitándolo de los listados y consultas por id.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Marca `activo = false`; la fila permanece en la base, nunca se borra físicamente.
- Solo puede darse de baja un evento que todavía esté `activo = true`; repetir la baja sobre el mismo evento responde 404, no 204: a diferencia de las transiciones de estado (bloquear/activar), este `DELETE` **no es idempotente**.

Respuesta esperada

- `204 No Content`, sin cuerpo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un evento vigente con ese identificador, o ya estaba dado de baja.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/sesiones/[id]/route.ts`

5. Reglas transversales de la API

Todas las rutas REST comparten el mismo pipeline: autenticación → control de caudal → validación de entrada → ejecución del SQL fijo → auditoría. Estas reglas aplican a cada requerimiento del documento y no se repiten en cada uno.

- **Envoltura de respuesta:** las respuestas exitosas devuelven `{ "data": ... }` y las fallidas `{ "error": { "code", "message", "details?" } }`.
- **Validación de entrada:** cada ruta define un esquema; los parámetros de query, el cuerpo JSON y los parámetros de ruta se combinan en un único objeto con precedencia query < body < path (un `{id}` de la URL siempre gana sobre un `id` del cuerpo).
- **Coerción de tipos:** los identificadores y montos que llegan por query o por path se convierten de texto a número automáticamente; los campos numéricos de un cuerpo JSON deben enviarse como número, no como texto, salvo que el requerimiento indique lo contrario.
- **Cuerpo JSON malformado:** un cuerpo que no parsea como JSON devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid JSON body.".
- **Errores de base de datos, mapeados por código SQLSTATE:** una violación de unicidad (`23505`, ej. un email o número de factura repetido) se traduce a `409 CONFLICT`; una violación de clave foránea o de restricción `CHECK` (`23503` / `23514`) se traduce a `400 VALIDATION_ERROR`; cualquier otro error de base de datos cae en `400 EXECUTION_ERROR`.
- `DELETE` **es siempre baja lógica:** actualiza `activo = false` (o `activa` en `cuentas`) y responde `204 No Content` **sin cuerpo** — no `200` con la fila afectada.

- **Verbo no soportado:** una ruta que no define el método HTTP usado responde `405 Method Not Allowed`; lo genera Next.js automáticamente, ninguna ruta lo arma a mano, y su cuerpo no sigue la envoltura `{ error }` del resto de la API.
- **Auditoría:** toda request, exitosa o fallida, deja un registro en `public.sql_audit_log` con la clave usada, el método y la ruta, la entrada validada, el resultado y la IP de origen. Un fallo de auditoría nunca convierte una respuesta exitosa en error.
- **Idempotencia de las transiciones de estado:** los endpoints que fijan un estado (`bloquear`, `activar`, `confirmar`, `cancelar`, `leer`) escriben el valor destino sin verificar el estado previo, por lo que repetir la llamada devuelve el mismo resultado.
- **Las transiciones de estado anteriores al CRUD no miran `activo`:** `PATCH .../bloquear`, `.../activar`, `.../leer`, `.../confirmar`, `.../cancelar`, `.../kyc` y `POST .../pagar` no filtran por `activo = true` (ninguno de esos archivos se tocó al agregar el CRUD). Un recurso dado de baja con el nuevo `DELETE` deja de aparecer en `GET`, pero **sigue pudiendo bloquearse, confirmarse, leerse o pagarse** a través de su acción dedicada — una inconsistencia real del sandbox, no un comportamiento a asumir como intencional.

Códigos de éxito

Status	Cuándo ocurre
`200 OK`	GET/PUT/PATCH exitoso; también un POST de upsert que reactivó una fila existente en vez de crearla.
`201 Created`	POST que crea una fila nueva.
`204 No Content`	DELETE exitoso (baja lógica) — la respuesta no lleva cuerpo.

Códigos de error

Código	HTTP	Significado
`UNAUTHORIZED`	401	Falta el header `x-api-key`, o la clave no existe o está inactiva.
`RATE_LIMITED`	429	Se superaron las 200 requests por minuto de esa API key.
`VALIDATION_ERROR`	400	La entrada no cumple el esquema de la ruta, el JSON es inválido, o Postgres rechazó una clave foránea/CHECK.
`EXECUTION_ERROR`	400	La consulta falló en la base de datos por un motivo no cubierto por los códigos anteriores.
`NOT_FOUND`	404	El recurso indicado no existe, ya está dado de baja (`activo=false`), o no está en un estado que admita la operación.
`CONFLICT`	409	Violación de restricción `UNIQUE` en Postgres (ej. email, número de factura o nombre de rol duplicado).
`INTERNAL_ERROR`	500	Fallo inesperado del servidor (por ejemplo, no se pudo verificar la API key).

6. Flujo en la aplicación web

La aplicación web consume exactamente los mismos endpoints a través del proxy `/api/proxy/{path}`, que agrega la API key del lado del servidor y reenvía los headers de límite de caudal. El menú de navegación muestra únicamente los módulos del grupo del alumno cuando su email figura en el roster del curso; si no figura, muestra los diez módulos. Los errores de la API se muestran en pantalla con el mensaje que devolvió el backend, y un `401` cierra la sesión local. **La interfaz web todavía no tiene pantallas para `PUT` / `DELETE` ni para los recursos nuevos (`sesiones`, `movimientos`)**: solo cubre los flujos de creación y consulta que ya existían; esos requerimientos se prueban llamando la API directamente (Postman, `curl`, o el cliente HTTP del framework de automatización), no navegando la web.

Pantalla	Qué permite hacer
`/login` — Acceso con API key	Formulario de una sola entrada donde el alumno pega su API key. La clave se valida contra la API antes de guardarse; solo si la validación pasa se almacena y se habilita la navegación.
`/auth/login` — Ingreso de usuario	Formulario de email que ejecuta RF-G1-01. Con el ingreso aceptado se guarda el usuario en la sesión local y se navega al inicio.
`/auth/login` — Bloque "Recuperar acceso"	Se despliega con el enlace "¿Olvidaste tu contraseña?". Primero ejecuta RF-G1-03 con un email; cuando obtiene respuesta habilita el botón de confirmación, que ejecuta RF-G1-04 y muestra el mensaje de reseteo completado.
Barra de navegación — "Cerrar sesión"	Ejecuta RF-G1-02 con el usuario en sesión y luego limpia la sesión local. Si la llamada al backend falla, la sesión local igual se cierra.

- El guard de la aplicación redirige a `/login` mientras no haya API key, y a `/auth/login` mientras no haya usuario de negocio; las rutas `/usuarios/*` son la única excepción a esta segunda barrera.
- El Grupo 1 no tiene un módulo propio en el menú: sus pantallas son las de acceso, comunes a todos los alumnos.
- Una respuesta 401 desde cualquier pantalla dispara el cierre de la sesión local y la vuelta a `/login`.

7. Criterios de aceptación

Criterios de aceptación redactados en prosa, uno o más por requerimiento, cubriendo el camino feliz y los caminos negativos. Son la base para derivar escenarios BDD.

RF-G1-01 — Iniciar sesión

- Dado un usuario activo del sandbox, al iniciar sesión con su email la respuesta es 200 y `data.email` coincide con el email enviado.
- Tras un login exitoso, existe una nueva fila en `sesiones` para ese usuario con `tipo_evento = 'login'` y `exitoso = true`.
- Al iniciar sesión con el email de un usuario inactivo, la respuesta es 400 con código `VALIDATION_ERROR` y el mensaje "Usuario no encontrado o inactivo.".
- Al iniciar sesión con un email que no existe, la respuesta es idéntica a la del usuario inactivo (mismo código, mismo mensaje) y no se crea ninguna fila en `sesiones`.
- Al enviar un email sin formato válido, la respuesta es 400 `VALIDATION_ERROR` y el detalle indica el campo `email`.

- Al llamar el endpoint sin header `x-api-key`, la respuesta es 401 `UNAUTHORIZED`, aunque el email sea válido.

RF-G1-02 — Cerrar sesión

- Dado un usuario existente, al cerrar sesión la respuesta es 200 y `data.tipo_evento` es `logout`.
- Al cerrar sesión dos veces seguidas para el mismo usuario, ambas respuestas son 200 y se crean dos filas distintas en `sesiones`.
- Al cerrar sesión con un `usuarioId` inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Usuario no encontrado.".
- Al enviar `usuarioId` con valor cero o negativo, la respuesta es 400 `VALIDATION_ERROR`.
- El cierre de sesión de un usuario inactivo es aceptado y devuelve 200, a diferencia del login.

RF-G1-03 — Solicitar recuperación de contraseña

- Dado un usuario registrado, al solicitar la recuperación la respuesta es 200 y `data.tipo_evento` es `password_reset_solicitado`.
- La respuesta incluye `data.usuario_id`, y ese valor permite completar el reseteo en el paso siguiente.
- Al solicitar la recuperación con un email inexistente, la respuesta es 404 `NOT_FOUND`; el contraste con el login (que devuelve 400 y no distingue) es una diferencia deliberada del sandbox.
- Un usuario inactivo puede solicitar la recuperación y obtiene 200.

RF-G1-04 — Completar el reseteo de contraseña

- Dado el `usuario_id` obtenido al solicitar la recuperación, al completar el reseteo la respuesta es 200 y `data.tipo_evento` es `password_reset_completado`.
- Al completar el reseteo sin haber solicitado antes la recuperación, la respuesta sigue siendo 200: el sistema no exige la solicitud previa.
- Al completar el reseteo con un `usuarioId` inexistente, la respuesta es 404 `NOT_FOUND`.
- Después de completar el reseteo, el usuario puede iniciar sesión con normalidad; el login no se ve afectado por el reseteo.

RF-G1-05 — Listar eventos de sesión

- Al listar sesiones filtrando por un titular, todos los elementos devueltos tienen ese `usuario_id`.
- Un evento dado de baja con RF-G1-09 deja de aparecer en este listado.
- Un login exitoso vía RF-G1-01 aparece en este listado sin necesidad de crearlo aparte.

RF-G1-06 — Crear un evento de sesión genérico

- Al crear un evento con datos válidos, la respuesta es 201 y el evento aparece en el listado de RF-G1-05.
- Al crear un evento con `exitoso=false`, la respuesta es 201: a diferencia de `/auth/login`, este endpoint sí permite registrar un evento fallido.
- Al crear un evento para un `usuarioId` inexistente, la respuesta es 400 `VALIDATION_ERROR`.
- Al enviar `tipoEvento=eliminado`, la respuesta es 400 `VALIDATION_ERROR`.

RF-G1-07 — Consultar un evento de sesión

- Dado un evento creado con RF-G1-06, al consultarlo por su `id` la respuesta es 200 y los datos coinciden con los enviados.
- Al consultar un evento ya dado de baja, la respuesta es 404 `NOT_FOUND`, igual que un identificador inexistente.

RF-G1-08 — Reemplazar los metadatos de un evento de sesión

- Dado un evento existente, al reemplazar su `ip` la respuesta es 200 y `data.ip` refleja el nuevo valor.
- Al reemplazar sin enviar `userAgent`, el campo queda vacío aunque antes tuviera un valor.
- Al reemplazar un evento inexistente o dado de baja, la respuesta es 404 `NOT_FOUND`.

RF-G1-09 — Dar de baja un evento de sesión

- Dado un evento vigente, al darlo de baja la respuesta es 204 sin cuerpo.
- Tras la baja, `GET /sesiones/{id}` sobre ese mismo evento responde 404.
- Al repetir la baja sobre el mismo evento, la segunda respuesta es 404 `NOT_FOUND`, no 204.

8. Anexo normativo (BCP / SEDECO) y brechas

> Este anexo es material de referencia orientativo para dar contexto de negocio a los escenarios de prueba. Las normas citadas del Banco Central del Paraguay (BCP) y de la Secretaría de Defensa del Consumidor y el Usuario (SEDECO) se mencionan de forma general y deben validarse con su texto vigente antes de usarse como criterio de cumplimiento. El sandbox es un entorno didáctico y no pretende cumplir ninguna de ellas.

Referencia normativa	Expectativa	Estado en el sandbox
Ley 1015/97 de prevención de lavado de dinero y sus modificatorias (marco supervisado por SEPRELAD y el BCP)	Identificación fehaciente del cliente antes de operar y trazabilidad de los accesos a los servicios financieros.	Parcial: existe bitácora de accesos con IP y fecha, pero la identificación se reduce a un email sin credencial ni segundo factor.
Marco del BCP sobre seguridad de canales electrónicos y banca digital	Autenticación robusta del usuario, control de intentos fallidos y bloqueo preventivo de la cuenta.	No implementado: no hay contraseña, ni conteo de intentos, ni bloqueo; los intentos rechazados ni siquiera se registran.
Ley 6534/2020 de protección de datos personales crediticios	Minimizar la exposición de datos personales y evitar que los mensajes de error confirmen la existencia de un titular.	Parcial: el login no distingue entre usuario inexistente e inactivo, pero la recuperación de contraseña sí devuelve 404 y revela si el email está registrado.
Ley 1334/98 de Defensa del Consumidor (SEDECO), deber de información	Mensajes de error claros y comprensibles para el usuario ante un rechazo de acceso.	Cubierto: la API devuelve mensajes en español y la web los muestra tal cual en el formulario correspondiente.

Brechas conocidas del sandbox

- No existe columna de contraseña ni almacenamiento de credenciales: el "login" es una validación de existencia y estado del usuario.

- Los intentos de acceso rechazados no se registran en `sesiones`, por lo que la bitácora no sirve para detectar ataques de fuerza bruta.
- El circuito de recuperación no emite token, no vence y no exige la solicitud previa: cualquier `usuarioId` válido puede marcarse como reseteado.
- `forgot-password` responde 404 ante un email desconocido, lo que permite enumerar usuarios registrados.
- La columna `user_agent` de `sesiones` existe pero `/auth/*` nunca la completa (sí lo hace el CRUD genérico).
- No hay cierre de sesión del lado del servidor: `logout` solo agrega una fila a la bitácora, sin invalidar nada.
- El CRUD genérico de `sesiones` (RF-G1-06/08) permite crear o editar eventos con `exitoso=false` o cualquier `ip` / `userAgent` sin que el evento haya ocurrido de verdad: no es una fuente confiable de auditoría, solo un ejemplo didáctico de CRUD.
- A diferencia de las transiciones de estado del resto de la API, `DELETE /sesiones/{id}` no es idempotente: repetirlo sobre el mismo evento da 404, no 204.